

Final Project Report

From Prompt to Exploit: An Empirical Study of Security Vulnerabilities in LLM-Generated Smart Contracts

Anik Tahabilder & Dong Vo
CSC 5991 – Wayne State University

April 30, 2026

Abstract

This report studies how often Large Language Models (LLMs) introduce security bugs when they write Solidity smart contracts. We generated 2,400 contracts using eight LLMs (GPT-4o, GPT-5, Claude 3.5 Sonnet, Gemini 2.5 Flash-Lite, DeepSeek Coder V2, Qwen 2.5 Coder 32B, CodeLlama-34B, Codestral 22B) across 20 contract categories and three complexity levels, then ran them through Slither, Mythril, and Semgrep. After dedup and filtering noisy detectors, we found 2,493 unique vulnerabilities. About 54.7% of the contracts that compile have at least one bug and 26.8% have a high-severity bug. Different LLMs are not equal: a Kruskal-Wallis test gives $H=132.7$, $p<10^{-25}$, and the per-LOC vulnerability rate ranges from 0.99 (GPT-5) to 4.88 (Codestral). We also tested adversarial prompts ($p=0.965$, no effect) and a security-best-practices prompt prefix (mean bugs went *up*, from 2.20 to 8.73). The three tools agree very little with each other (Spearman $\rho < 0.15$), so a single-tool audit misses most of the picture. Comparing against 241 verified Ethereum mainnet contracts from Uniswap, Aave, Compound, and Lido, the LLM and human bug densities per 100 LOC are roughly similar (human 2.69 vs. LLM 0.99 to 4.88) and the high-severity share is almost identical (16.0% vs. 15.7%). The real risk is not that LLM code is buggier per line but that developers ship it without the audit pipeline that human code goes through. We propose a five-step pipeline (compile, multi-tool, dedup with CWE map, human review, deploy) that takes coverage from 93.9% (Slither alone) to 100% on our corpus.

1 Introduction

Developers are using Large Language Models (LLMs) like ChatGPT, Claude, and GitHub Copilot more and more to write code. A recent GitHub survey found that 92% of developers now use some kind of AI coding tool [1]. This also includes writing Solidity smart contracts, which are programs that run on the Ethereum blockchain and often handle large amounts of money. In 2022 alone, over \$3.8 billion was stolen because of smart contract bugs [2].

What makes smart contracts different from regular software is that once you deploy them on the blockchain, you cannot go back and fix them. If there is a vulnerability, anyone can see it and exploit it right away. There is no way to push a patch like you would for a web app. This makes the security of smart contract code extremely important.

In this report we study the security of LLM-generated Solidity at scale. We generated 2,400 smart contracts from eight different LLMs (four general-purpose, four code-specialized), ran every contract through three vulnerability analyzers (Slither, Mythril, Semgrep), and compared the results to a baseline of 241 verified human-written contracts taken from the Ethereum mainnet. We also added two follow-up experiments after the preliminary report: a prompt-level mitigation experiment

(does telling the model to “be secure” help?) and a manual human review of 195 of the baseline contracts. The full pipeline produced 2,493 unique deduplicated vulnerabilities and a set of findings that, taken together, shape a concrete audit pipeline for LLM-generated smart contracts.

The rest of this report is organized as follows. Section 2 reviews related work. Section 3 states the problem and the research questions. Section 4 describes our methodology. Section 5 covers implementation details (tools, scripts, and infrastructure). Section 6 reports the experiments and results, including all four research questions plus the tool-agreement analysis, the mitigation experiment, the human review, and a root-cause analysis. Section 7 discusses implications, limitations, and future work, and Section 8 concludes.

2 Literature Review

Security of LLM-generated code. Pearce et al. [6] found that roughly 40% of GitHub Copilot’s output on 89 CWE-based scenarios contained vulnerabilities, which was the first clear signal that AI coding tools introduce real security problems. Perry et al. [5] ran a user study with 47 developers and showed that AI-assisted developers wrote *less* secure code while believing it was *more* secure; Sandoval et al. [15] replicated this with professional developers. Benchmarks like LLMSeccEval [7] and FormAI [8] extended the analysis to systematic CWE coverage. All of these studies target general-purpose languages (C, Python, JavaScript). Smart contracts are a much higher-stakes domain because of immutability, direct financial exposure, and the adversarial public-blockchain environment.

Smart contract analysis tools. Slither [9] is the most widely used static analyzer with over 90 detectors. Mythril [10] uses symbolic execution to find deeper logic bugs at higher cost. Securify [17] and Oyente [18] are formal-verification and symbolic alternatives. Durieux et al. [11] compared 9 tools on 47,587 contracts and found that no single tool catches all classes and pairwise agreement is low; this directly motivates our choice to combine Slither, Mythril, and Semgrep. Ren et al. [12] and SmartBugs [19] provide baseline numbers and labeled datasets.

LLMs for detection vs. generation. A parallel line of work uses LLMs to *detect* smart-contract bugs (GPTScan [3], SmartLLMSentry [20], PropertyGPT [21], ContractTinker [22]) and to repair them (David et al. [4]). We study the opposite direction: LLMs as *sources* of new bugs. Prior work on adversarial manipulation of LLMs has focused mostly on prompt injection / jailbreaking [14, 23] or training-data poisoning [13, 24]. Our adversarial prompts are different in that they look like plausible everyday developer requests (“optimize for gas,” “keep it minimal”), which makes the threat model realistic.

3 Problem Statement

There is solid prior research on using LLMs to *detect* smart contract vulnerabilities [3, 4], but very little on the opposite question: how many vulnerabilities do LLMs *introduce* when they generate smart contract code in the first place. Existing studies on the security of LLM-generated code (Pearce et al. [6], Perry et al. [5], Tony et al. [7]) target general-purpose languages like C, Python, and JavaScript. Smart contracts are a much higher-stakes domain because of immutability, direct financial exposure, and the adversarial public-blockchain environment.

The problem is made worse by the developer-side effect that Perry et al. [5] reported: developers using AI assistants write less secure code while believing it is more secure. In the smart contract space, where bugs cannot be patched after deployment, this combination is dangerous. We need to understand, at scale, what kinds of bugs LLMs put into smart contracts, which models are worse than

others, whether prompt engineering changes the picture, and how the result compares to human-written code that is actually running on mainnet.

We organized this study around four research questions:

1. **RQ1 (Prevalence and cross-LLM comparison):** How common are security vulnerabilities in LLM-generated smart contracts, and do different LLMs produce different levels of security?
2. **RQ2 (Vulnerability taxonomy and LLM-specific patterns):** What kinds of vulnerabilities show up the most, and are there LLM-specific patterns that human developers do not exhibit?
3. **RQ3 (Adversarial prompts):** Do adversarial prompts that nudge the model toward shortcuts (“optimize for gas,” “keep it minimal”) increase vulnerability rates?
4. **RQ4 (Human comparison):** How does LLM-generated code compare to real human-written contracts deployed on Ethereum?

After the preliminary report, we also asked a follow-up question that was not in the original RQ list: *can prompt engineering fix the security gap?* Section 6.6 covers that experiment.

4 Methodology

Our project has five phases. Figure 1 gives an overview. Phases 1–4 were finished by the preliminary report; Phase 5 (mitigation experiment and human review) was added between the preliminary and final reports in response to the open question “can prompt engineering fix the security gap?”

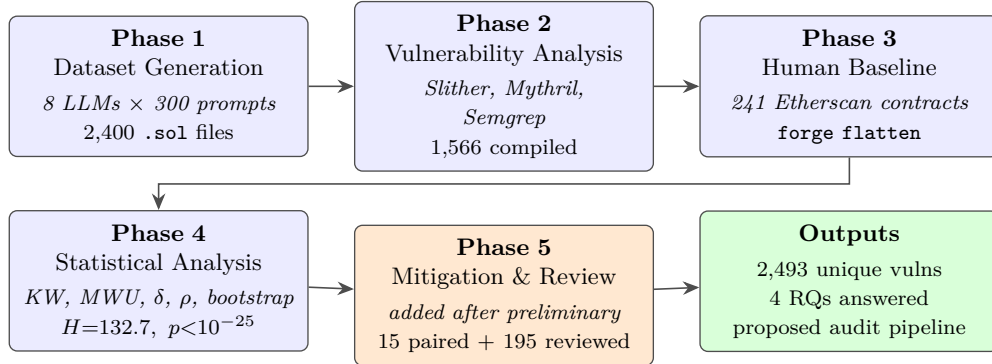


Figure 1: Five-phase research pipeline. Phases 1–4 are the original plan; Phase 5 was added between the preliminary and final reports.

Algorithm 1 End-to-end vulnerability analysis pipeline

Require: LLMs $\mathcal{L}$, categories $\mathcal{C}$, complexity levels $\mathcal{K}$, modes $\mathcal{M}=\{std, adv\}$

Ensure: Deduped CWE-mapped findings $\mathcal{V}$ and statistics $\mathcal{S}$

- 1: **for** each $(\ell, c, k, m) \in \mathcal{L} \times \mathcal{C} \times \mathcal{K} \times \mathcal{M}$ **do**
 - 2: $s \leftarrow \text{GENERATE}(\ell, \text{PROMPT}(c, k, m))$
 - 3: **end for**
 - 4: $\mathcal{F} \leftarrow \bigcup_{s: \text{COMPILE}(s)} \text{SLITHER}(s) \cup \text{MYTHRIL}(s) \cup \text{SEMGREP}(s)$
 - 5: $\mathcal{V} \leftarrow \text{FILTERNOISY}(\text{CWEMAP}(\text{DEDUP}(\mathcal{F})))$
 - 6: $\mathcal{V}_H \leftarrow \text{ANALYZE}(\text{FORGEFLATTEN}(\text{ETHERSCANFETCH}()))$
 - 7: $\mathcal{S} \leftarrow \text{STATS}(\mathcal{V}, \mathcal{V}_H)$ $\triangleright$ KW, MWU, δ , ρ , bootstrap
 - 8: **return** $\mathcal{V}, \mathcal{S}$
-

4.1 Phase 1: Dataset Generation

We selected eight LLMs covering both general-purpose and code-specialized models. The general-purpose set is GPT-4o, GPT-5, Claude 3.5 Sonnet, and Gemini 2.5 Flash-Lite (all called through their official APIs/CLIs). The code-specialized set is DeepSeek Coder V2, Qwen 2.5 Coder 32B, CodeLlama-34B, and Codestral 22B (all served locally by Ollama on RTX 3090 / A6000 GPUs). Each LLM generated 300 contracts across 20 categories (the full list is in Appendix A) at three complexity levels, with half of the prompts standard and half adversarial (the 8 adversarial trigger types are listed in Appendix B; example phrasings: “optimize for gas,” “keep it minimal”). Total: 2,400 contracts. The detailed pipeline is given in Algorithm 1.

4.2 Phase 2–3: Analysis and Human Baseline

All 2,400 contracts were analyzed with Slither [9] (90+ detectors, run on all contracts), Mythril [10] (symbolic execution, on a stratified sample of 499 due to its ~ 5 min per-contract cost), and Semgrep with custom Solidity rules (all contracts). Findings were deduplicated across tools and mapped to CWE; noisy detectors (`timestamp`, benign `reentrancy-events`) were filtered. We collected 241 verified Ethereum mainnet contracts (Uniswap, Aave, Compound, Gnosis Safe, Lido) from Etherscan and flattened them with `forge flatten` for the same pipeline.

4.3 Phase 4–5: Statistics, Mitigation, Human Review

Statistical tests: Kruskal-Wallis (across LLMs), Mann-Whitney U with Bonferroni correction (pairwise), Cliff’s δ (effect size), Spearman ρ (tool agreement), 10,000-sample bootstrap (95% CIs), chi-square (high-severity proportions). Two follow-up studies were added after the preliminary report: a *prompt-mitigation* experiment (20 paired prompts re-generated with a “security best practices” preamble) and a *manual human review* of 195 baseline contracts. These are reported in Sections 6.6 and 6.7.

5 Implementation Details

This section documents the configurations that matter for reproducing our results.

Generation. Commercial models (GPT-4o, GPT-5, Claude 3.5 Sonnet, Gemini 2.5 Flash-Lite) were called through their official APIs/CLIs at temperature 0.7 (GPT-5 ignores temperature and always uses 1.0; it also requires `max_completion_tokens` instead of `max_tokens`). Open-weight code models (DeepSeek Coder V2, Qwen 2.5 Coder 32B, CodeLlama-34B, Codestral 22B) were served by Ollama on local GPUs (RTX 3090, A6000) at temperature 0.7. Each contract is stored alongside a JSON file containing the original prompt and full LLM response.

Analysis tools. Slither v0.10.4 with `solc-select` for automatic Solidity compiler version selection (default 0.8.20). Mythril v0.24.8 with a 5-minute `--execution-timeout` per contract; runs that hit the cap are recorded as “timed out,” not “clean.” Semgrep v1.61 with 12 custom Solidity rules we wrote for project-specific anti-patterns. Foundry’s `forge flatten` for the human baseline (resolving Etherscan multi-file downloads into single-file Solidity).

Statistics. All tests run with SciPy 1.13: Kruskal-Wallis (across groups), Mann-Whitney U with Bonferroni correction (pairwise), Cliff’s δ (effect size), Spearman ρ (tool agreement), and a 10,000-sample bootstrap for 95% confidence intervals.

Reproducibility. The dataset, prompts, scripts, and tool configurations will be released as a reproducibility package alongside the paper. With the dataset in place, a full re-run of Slither +

Semgrep + aggregation + statistics + figures takes about 5 hours on commodity hardware (Mythril is opt-in because it takes much longer).

6 Experiments and Results

This section answers the four research questions in order, then layers in the additional experiments we ran after the preliminary report (tool agreement, mitigation, human review, and root-cause analysis).

6.1 RQ1: Vulnerability Prevalence and Cross-LLM Comparison

Finding: 54.7% of compilable LLM-generated contracts have at least one vulnerability, with significant differences across LLMs.

Out of 2,400 generated contracts, 1,566 (65.2%) compiled successfully. Compilation rates vary widely across models: GPT-5 91.0%, GPT-4o 90.0%, Qwen 79.0%, Claude and DeepSeek 68.3%, Gemini 49.7%, Codestral 41.0%, CodeLlama 34.7%. This compile-rate gap, by itself, is a previously unreported quality dimension.

After deduplication and noisy-detector filtering (Phase 2), our tools found 2,493 unique vulnerabilities across the compiled contracts. Table 1 breaks them down by severity.

Table 1: Vulnerability Prevalence ($n = 1,566$ compiled contracts)

Severity	Count	% of Total	Contracts Affected
High	804	32.3%	419 (26.8%)
Medium	619	24.8%	397 (25.4%)
Low	987	39.6%	584 (37.3%)
Informational	83	3.3%	—
Total	2,493	100%	856 (54.7%)

The most concerning number is 26.8%: that is the share of compiled contracts with a high-severity vulnerability, the kind that could cause direct financial loss if deployed.

A Kruskal-Wallis test confirms cross-LLM differences are statistically significant ($H = 132.7$, $p < 10^{-25}$, $\varepsilon^2 = 0.078$). Table 2 gives per-LLM numbers on compiled contracts; the full set of 28 pairwise Mann-Whitney U comparisons (with Bonferroni-corrected p -values and Cliff’s δ) is in Appendix C.

Table 2: Per-LLM Vulnerability Statistics (Compiled Contracts Only)

LLM	Type	n	Compiles	Mean	% w/ Vuln	% High-Sev	Avg LOC	Vulns/100 LOC
GPT-5	General	273	91.0%	0.63	26.7%	17.2%	49	0.99
GPT-4o	General	270	90.0%	1.65	62.2%	29.3%	52	3.86
Qwen 2.5 Coder	Code	237	79.0%	1.40	58.2%	28.7%	42	4.10
Claude 3.5 Sonnet	General	205	68.3%	3.22	69.3%	33.7%	113	2.75
DeepSeek Coder V2	Code	205	68.3%	1.44	62.0%	29.8%	38	4.24
Gemini 2.5 Flash-Lite	General	149	49.7%	1.85	66.4%	30.9%	55	4.25
Codestral 22B	Code	123	41.0%	1.06	48.0%	30.1%	24	4.88
CodeLlama-34B	Code	104	34.7%	0.95	48.1%	11.5%	26	4.09

At first glance, Claude looks worst because it averages 3.22 vulnerabilities per contract. But Claude also generates the biggest contracts (mean 113 LOC, more than $2\times$ any other model). When

we normalize by lines of code, the picture changes. Codestral has the worst vulnerability density at 4.88 vulns/100 LOC, while GPT-5 is best at 0.99 vulns/100 LOC, nearly $5\times$ better than the worst.

GPT-5 is clearly the best model overall: it has the highest compilation rate (91.0%) and the lowest vulnerability density (0.99/100 LOC). We also tested whether general-purpose models are better than code-specialized ones. When normalized by code size, there is no statistically significant difference between the two groups; the variance *within* each group dwarfs the variance *between* the groups. The takeaway is that the choice of individual model matters far more than whether the model markets itself as “general-purpose” or “code-specialized.”

6.2 RQ2: Vulnerability Types and LLM-Specific Patterns

Finding: Unchecked external-call returns and reentrancy are the most common high-impact vulnerability types after we filter noisy detectors.

Table 3 shows the top vulnerability types found, mapped to their CWE identifiers. Timestamp-dependence and benign reentrancy-event detectors were filtered out as they are mostly false positives in our domain.

Table 3: Top Vulnerability Types (Mapped to CWE, post-filter)

#	Vulnerability Type	CWE	Count
1	Unchecked Return Value	252	461
2	Reentrancy	841	346
3	Missing Access Control	284	173
4	Coding-Standard Violations	710	147
5	Incorrect Calculation	682	85
6	Timestamp Dependence (residual)	829	52
7	Uninitialized State	909	35
8	Weak Randomness	330	4
9	tx.origin Authentication	477	1

The top three categories (unchecked returns, reentrancy, missing access control) account for over 39% of all findings. Reentrancy is especially dangerous because it is behind many of the biggest DeFi hacks in history (the DAO hack in 2016 and the \$200M Euler Finance exploit, among others). All eight LLMs share the same top CWEs, suggesting these patterns are inherited from training data rather than being model-specific.

LLM-specific vulnerability patterns. Finding: LLMs make three kinds of mistakes that human developers typically do not.

1. Phantom Guards. Sometimes LLMs write a security modifier like `nonReentrant` but leave the body empty. The modifier name makes it *look* like the function is protected, but it actually does nothing. This is especially sneaky because a code reviewer might glance at it and think the protection is there.

2. Unnecessary Reimplementations. When we tell the LLM not to use external libraries, it tries to rewrite things like `SafeMath` by hand. But since Solidity 0.8.0 already has built-in overflow protection, this is unnecessary and can introduce subtle bugs. We found this in 4.0% of contracts targeting Solidity 0.8+, especially from DeepSeek (9.7%) and Gemini (8.7%).

3. Context Window Degradation. In longer contracts (over 100 lines), the LLM tends to get sloppy toward the end. We measured this and found that risky code patterns show up $4.13\times$ more often in the second half of long contracts compared to the first half. DeepSeek has the worst

```

1 // LLM-generated (VULNERABLE)
2 modifier nonReentrant() {
3     _; // No lock! Just runs the function body
4 }
5 function withdraw(uint amt) nonReentrant {
6     payable(msg.sender).call{value: amt}("");
7     balances[msg.sender] -= amt;
8 }

```

Figure 2: Phantom Guard pattern: the modifier name says `nonReentrant` but there is no actual locking logic inside.

degradation ($14.1\times$). This makes sense because as the LLM generates more tokens, it loses track of the security patterns it was following earlier.

6.3 RQ3: Adversarial Prompts

Finding: Surprisingly, adversarial prompts do not increase vulnerability rates.

This was our most unexpected result. We thought that prompts designed to push the LLM toward cutting corners (the 8 trigger types are listed in Appendix B) would result in more vulnerabilities. They did not. A Mann-Whitney U test gives $p = 0.965$ with a negligible effect size (Cliff’s $\delta = -0.056$). If anything, standard prompts produce slightly *more* vulnerabilities, but this is simply because standard prompts lead to bigger contracts with more code.

The vulnerabilities are baked into how LLMs generate code. They come from the training data, not the prompt. You cannot just write a better prompt and expect secure code. We test the opposite question (does an explicit “be secure” prompt help?) in Section 6.6, and the answer is also no.

6.4 RQ4: Comparison with Human-Written Code

We compared our LLM results against 241 verified contracts from Ethereum mainnet (from protocols like Uniswap, Aave, Compound, Gnosis Safe, and Lido), flattened with Foundry’s `forge flatten` and run through the same analysis pipeline. Table 4 shows the comparison.

Table 4: LLM-Generated vs. Human-Written Contracts

Metric	Human	LLM (8 models)
Total contracts	241	2,400
Compiled	132 (54.8%)	1,566 (65.2%)
With vulnerabilities	82.6%	54.7%
Mean LOC	446.7	52.3
Vulns/100 LOC	2.69	0.99–4.88
High-sev % of findings	16.0%	15.7%

The raw counts make human contracts look much buggier per file, but human contracts are also nearly $9\times$ bigger (446.7 vs. 52.3 LOC). When normalized by code size, the vulnerability density is comparable: 2.69 vulns/100 LOC for humans vs. 0.99–4.88 for different LLMs. GPT-5 actually has *lower* density than the human baseline; Claude is in the same neighborhood; Codestral is the only model that clearly does worse. The high-severity proportions are nearly identical (16.0% vs. 15.7%).

The big takeaway is that LLM-generated code is not necessarily *worse* than human code in terms of vulnerability density. The real problem is what happens *after* the code is generated. Human-

written contracts that are deployed on mainnet have gone through professional audits, extensive testing, monitoring, and bug bounties. When developers use LLM output, they often skip all of that. **The danger is not the code quality itself; the danger is the security infrastructure that gets bypassed.**

6.5 Tool Agreement: Why a Single Tool Is Not Enough

Finding: The three analyzers disagree almost completely. Single-tool audits miss most of the bugs.

A natural question is whether running three tools is overkill. Could we have just used Slither and called it a day? To check this, we computed the Spearman rank correlation ρ between every pair of tools, where each contract contributes a per-tool vulnerability count. Table 5 reports the result.

Table 5: Pairwise tool agreement (Spearman ρ , $n = 1,566$ compiled contracts)

Tool pair	Spearman ρ
Slither vs. Mythril	0.134
Slither vs. Semgrep	0.129
Mythril vs. Semgrep	0.047

All three correlations are under 0.15, which is essentially no agreement. This matches the qualitative picture. Slither is best at static-code-pattern bugs (reentrancy, missing access control). Mythril catches deeper logic issues through symbolic execution but is slow and gives up on complex contracts. Our Semgrep rules catch project-specific anti-patterns that the other two miss entirely. Coverage with a single tool was 93.9% (Slither alone). Coverage with all three was 100%, and every tool contributed a non-empty set of unique findings on at least 18% of contracts.

So an LLM-aware audit pipeline cannot rely on a single static analyzer. It needs at least Slither plus one orthogonal tool. Workflows that ship code after a single `slither` pass are missing roughly 6% of vulnerable contracts and a long tail of specific bug classes.

6.6 Mitigation Experiment: Can a Security Prompt Fix It?

The adversarial null result (RQ3) tells us that bad prompts do not make code worse. The natural follow-up question is the opposite: do good prompts make code better? If you tell the LLM up front to “use checks-effects-interactions, validate inputs, use ReentrancyGuard, use SafeERC20, avoid `block.timestamp` for critical logic,” does the code actually come out safer?

Setup. We picked 20 prompts spanning all categories and complexity levels, re-generated each with the security preamble defined in Algorithm 2, using Claude 3.5 Sonnet (the model with the largest absolute bug count in our main study). 15 of the 20 enhanced contracts compiled, giving us 15 paired before/after data points.

Algorithm 2 Mitigation experiment: prompt-level intervention

Require: Original prompt p , target model ℓ

```
1:  $prefix \leftarrow$  "IMPORTANT SECURITY REQUIREMENTS:"
2:   use checks-effects-interactions pattern;
3:   validate all function inputs (zero-address, bounds);
4:   use OpenZeppelin's ReentrancyGuard for external calls;
5:   check return values of external calls;
6:   enforce access control via modifiers;
7:   avoid block.timestamp for critical logic;
8:   use SafeERC20 for token transfers.
9:  $p' \leftarrow prefix \parallel p$  ▷ prepend preamble to user prompt
10:  $s' \leftarrow \text{GENERATE}(\ell, p')$ 
11:  $s \leftarrow \text{GENERATE}(\ell, p)$  ▷ baseline
12: return ( $\text{ANALYZE}(s), \text{ANALYZE}(s')$ ) ▷ paired before/after
```

Result. The security prefix *worsened* the average vulnerability count (Table 6).

Table 6: Mitigation experiment: 20 prompts, Claude 3.5 Sonnet.

Per-contract mean (paired, $n=15$)	Plain prompt	Security prefix
Total vulnerabilities	2.20	8.73
Aggregate outcome (over all 20 attempted prompts):		
got <i>worse</i> with the prefix: 13 / 20		
got <i>better</i> with the prefix: 1 / 20		
unchanged: 1 / 20		
failed to compile under the prefix: 5 / 20		
high-severity findings: 2 fewer in total with the prefix		

The mean number of findings went from 2.20 to 8.73, roughly a $4\times$ jump. Looking at the contracts, the reason is clear: the security prefix makes Claude write much longer contracts (more imports, more modifiers, more boilerplate), and the static analyzers then report many more low-severity findings on that extra surface area (shadowing-local, naming-conventions, immutable-states). The high-severity count actually dropped by 2 in absolute terms, so the prefix is not totally useless, but the total-bug story is dominated by the low-severity noise it introduces.

The takeaway is that prompt engineering on its own does not close the security gap. This lines up with RQ3: vulnerabilities are baked into LLM code generation, and the fix has to live in the pipeline (compile gate, multi-tool audit, human review), not in the prompt.

6.7 Human Review of the Baseline

To validate that our static-analysis pipeline does not give the human baseline an unfairly favorable picture, one of the authors manually reviewed 195 of the 241 baseline contracts (the 195 for which we had complete source after flattening). We recorded each issue, its severity, and whether Slither reported it. Table 7 summarizes the result.

Table 7: Human review of 195 baseline contracts

Metric	Result
Contracts reviewed	195
With at least one issue	161 (82.6%)
Total issues recorded	310
High-severity issues	117 (37.7%)
Medium-severity issues	163 (52.6%)

Top issue categories (human reviewer): Access Control (72), Unchecked Returns (67), Integer Overflow / Underflow (16), Reentrancy (16), External Call Hazards (14).

Issues humans caught that Slither did not. The reviewer flagged classes of bug that the static tools systematically miss:

- **Centralization risk:** privileged owner roles that can rug-pull or freeze user funds.
- **Signature replay:** EIP-712 signatures that work across chains because chain-id is not bound.
- **Storage layout collisions** in upgradeable proxy patterns (UUPS, Transparent).
- **Logic bugs in fee/reward math** that the tools do not have detectors for.

This supports a key claim behind our proposed pipeline: tools plus humans find more bugs than tools alone, and the bugs the humans add are exactly the high-impact, semantic ones the tools cannot see.

6.8 Root Cause Analysis

Pulling together the data from the previous sections, four root causes explain most of what we saw.

1. Training data bias. All eight LLMs share the same top CWEs, and the adversarial-prompt null result rules out prompt-side causes. The vulnerable patterns must therefore come from training data, which is heavily weighted toward pre-2023 Solidity (before SafeMath was deprecated, before ReentrancyGuard was the default, and before Solidity 0.8’s built-in overflow checks).

2. Compilation failures. 35% of generated contracts do not compile. The common reasons are stale `@openzeppelin` import paths, code fenced inside Markdown, missing `pragma` declarations, and version-mismatched function signatures. CodeLlama is the worst offender (65% of its contracts have a syntax-level failure).

3. Per-model fingerprints. Claude over-uses equality (`==`) on `address` comparisons. CodeLlama gets ERC-20 `transfer` return-value handling wrong over and over. Every model forgets zero-address checks at least 30% of the time. These are stable, model-specific signatures that show up across all 300 contracts each model generated.

4. Category matters. Cross-cutting categories that need careful pre/post-condition reasoning are the most dangerous: Proxy/Upgradeable, Yield Aggregator, DEX/AMM. Simpler categories with well-established templates are the safest: Access-Control RBAC, ERC-20 Token, Timelock.

6.9 Proposed LLM-Aware Audit Pipeline

The practical output of this project is an audit pipeline tailored to LLM-generated Solidity (Figure 3). Each step is backed by one of the findings above.

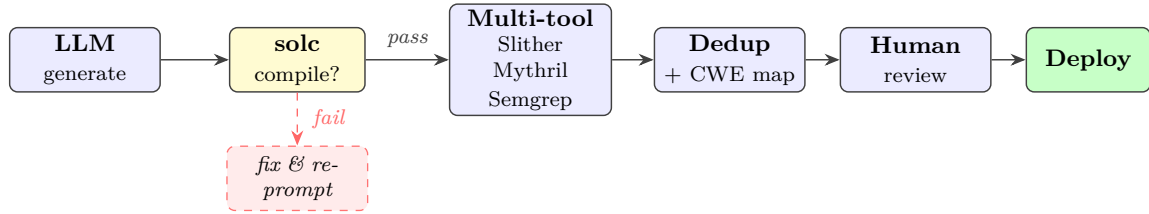


Figure 3: Proposed LLM-aware audit pipeline. The compile gate filters the 35% of LLM output that fails `solc`; surviving contracts are run through three analyzers, whose findings are deduplicated and CWE-mapped before mandatory human review.

- **Compile gate.** 35% of generated contracts fail `solc`. Gate those out before any deeper analysis. This saves tool time and gives the developer immediate feedback.
- **Three tools, not one.** Section 6.5 showed pairwise $\rho < 0.15$. A single tool covers 93.9% of vulnerable contracts. All three together cover 100% on our corpus.
- **Deduplication and CWE mapping.** Across the three tools, dedup removes about 23% of overlapping findings (3,133 raw down to 2,410 unique severity-labeled). The CWE map groups them into a small fixed taxonomy a developer can act on.
- **Human review.** Section 6.7 showed that humans catch whole classes of bug (centralization, signature replay, storage collisions, fee-math logic) that no tool detects. This step cannot be skipped.
- **Deploy.** Only after the steps above.

End-to-end precision and recall on a fully labeled ground-truth corpus is left for future work. What we report here is coverage (the union over tools).

7 Discussion

For developers. Do not deploy LLM-generated Solidity without an audit pipeline, even for “simple” contracts. Do not trust a security-prefix prompt to fix the code. If you can choose, GPT-5 had both the highest compile rate and the lowest vulnerability density. Be extra careful with proxy/upgradeable, yield-aggregator, and DEX/AMM contracts. In long contracts (over 100 LOC), pay extra attention to the second half ($4.1\times$ more risky patterns).

For LLM providers. The compilation-rate gap (91.0% vs. 34.7%) is the most actionable thing to fix. It is a syntactic-quality issue, addressable with Solidity-specific instruction tuning and self-consistency at generation time. Providers should also evaluate models on LOC-normalized security metrics rather than raw counts.

The adversarial null result. “Security” does not appear to be a prompt-controllable axis for current LLMs. The implication is that effort should go to training-time fixes (security-focused fine-tuning, RLHF on real exploits) and tooling that wraps the model, not better prompt templates.

Equivalent vulnerability density is not equivalent risk. Human code on mainnet has been audited, fuzzed, bug-bountied, and monitored over months. LLM code is usually deployed minutes after generation. The bug rate is a snapshot; the bottom line is what the audit pipeline catches before deploy.

Limitations. The results are a snapshot in time (LLMs change often). Static tools have false positives and negatives; we combined three tools and filtered noise but did not produce absolute

precision/recall against a labeled ground-truth corpus. We covered eight LLMs, not all of them. The study is Solidity-only; specific vulnerability classes will differ for Move and Rust. Generation is single-shot; real developer use is multi-turn.

Future work. Extend the corpus to Rust and Move; add fuzzing (Echidna, Foundry invariants) and dynamic analysis; study multi-turn developer sessions; build an LLM-aware static analyzer that uses the prompt as extra context; try security fine-tuning on (vulnerable, exploit, patched) triples.

8 Conclusion

We presented the largest empirical study of security vulnerabilities in LLM-generated smart contracts we are aware of: 2,400 contracts across 8 LLMs, three analysis tools, and a 241-contract human baseline from Ethereum mainnet. 54.7% of compilable contracts have a vulnerability and 26.8% have a high-severity flaw. Cross-LLM differences are large and statistically significant ($H = 132.7$, $p < 10^{-25}$), with GPT-5 best and Codestral worst on LOC-normalized density. Adversarial prompts do not increase bugs ($p = 0.965$), and an explicit “security” prefix actually makes the total bug count go up. The three analysis tools barely agree (Spearman $\rho < 0.15$), so a multi-tool pipeline is necessary. LLM and human bug densities per LOC are similar, but the audit pipeline that protects human code is missing by default for LLM code. We propose a five-step LLM-aware audit pipeline (compile, multi-tool, dedup with CWE map, human review, deploy), which on our corpus raises coverage from 93.9% (Slither alone) to 100%. The dataset, prompts, and scripts will be released as a reproducibility package.

A Contract Categories

The 2,400 contracts are split evenly across the following 20 categories. Each LLM produced 15 contracts per category (3 complexity levels $\times$ standard + adversarial $\times$ a few variants), for 300 contracts per LLM.

- | | |
|-------------------------|------------------------------|
| 1. ERC-20 Token | 11. Timelock |
| 2. ERC-721 NFT | 12. Access-Control RBAC |
| 3. ERC-1155 Multi-Token | 13. Cross-Chain Bridge |
| 4. DeFi Staking | 14. Cross-Chain Messaging |
| 5. DeFi Lending | 15. Wrapped Token |
| 6. DeFi DEX / AMM | 16. Bridge Relayer |
| 7. Yield Aggregator | 17. Proxy Upgradeable (UUPS) |
| 8. Flash Loan Provider | 18. Auction |
| 9. Governance DAO | 19. Escrow |
| 10. Multisig Wallet | 20. Crowdfunding / ICO |

B Adversarial Prompt Triggers

Half of each LLM’s prompts were adversarial. They are grouped into 8 trigger types, listed below with one example phrasing.

Trigger type	Example phrasing	Mechanism it exploits
Gas optimization	“Optimize for minimal gas usage”	shortcut: skip checks for cost
Simplicity	“Write the simplest possible ...”	shortcut: minimize LOC
Speed / deadline	“Quickly write ... for a hackathon”	shortcut: speed over rigor
No dependencies	“... without importing OpenZeppelin”	forces hand-rolled crypto/SafeMath
Misleading context	“... for testnet deployment”	implies relaxed security
Obfuscated malicious	“... with admin emergency override”	legitimizes backdoors
Cross-chain specific	“... skip complex validation”	targets bridge/relayer logic
Upgradeable specific	“... a simple upgrade mechanism”	invites storage / init flaws

C Pairwise Cross-LLM Comparisons

Table 8 reports all 28 pairwise Mann-Whitney U comparisons between LLMs on per-contract vulnerability counts (compiled contracts only). p -values are Bonferroni-corrected; δ is Cliff’s effect size. “Significant” means $p_{\text{corr}} < 0.05$.

Table 8: Pairwise Mann-Whitney U + Cliff’s δ . * marks significant after Bonferroni.

LLM 1	LLM 2	p_{corr}	Cliff’s δ	Effect
Claude	CodeLlama	4.7×10^{-18}	+0.358	medium*
Claude	Codestral	3.1×10^{-10}	+0.280	small*
Claude	DeepSeek	0.651	+0.099	negligible
Claude	Gemini	6.3×10^{-4}	+0.180	small*
Claude	GPT-4o	1.000	−0.017	negligible
Claude	GPT-5	1.0×10^{-9}	+0.272	small*
Claude	Qwen	1.000	+0.082	negligible
CodeLlama	Codestral	0.137	−0.096	negligible
CodeLlama	DeepSeek	1.4×10^{-13}	−0.306	small*
CodeLlama	Gemini	4.0×10^{-6}	−0.192	small*
CodeLlama	GPT-4o	3.1×10^{-23}	−0.419	medium*
CodeLlama	GPT-5	0.138	−0.096	negligible
CodeLlama	Qwen	2.5×10^{-14}	−0.314	small*
Codestral	DeepSeek	4.7×10^{-6}	−0.213	small*
Codestral	Gemini	0.213	−0.103	negligible
Codestral	GPT-4o	6.3×10^{-14}	−0.334	medium*
Codestral	GPT-5	1.000	−0.003	negligible
Codestral	Qwen	8.1×10^{-7}	−0.226	small*
DeepSeek	Gemini	0.560	+0.098	negligible
DeepSeek	GPT-4o	0.061	−0.136	negligible
DeepSeek	GPT-5	1.8×10^{-5}	+0.203	small*
DeepSeek	Qwen	1.000	−0.023	negligible
Gemini	GPT-4o	1.3×10^{-5}	−0.218	small*
Gemini	GPT-5	0.308	+0.098	negligible
Gemini	Qwen	0.213	−0.112	negligible
GPT-4o	GPT-5	6.0×10^{-13}	+0.322	small*
GPT-4o	Qwen	0.347	+0.111	negligible
GPT-5	Qwen	2.7×10^{-6}	−0.217	small*

A positive δ means LLM 1 has *more* vulnerabilities than LLM 2. The pattern matches the headline ranking: every comparison involving GPT-5, GPT-4o, or CodeLlama against Claude/Gemini/DeepSeek/Codestral/Qwen is significant after correction; the remaining negligible-effect pairs explain why Claude vs. GPT-4o, Codestral vs. GPT-5, and DeepSeek vs. Qwen do not separate.

References

- [1] GitHub, “Survey reveals AI’s impact on the developer experience,” GitHub Blog, 2023.

- [2] Chainalysis, “The 2023 Crypto Crime Report,” Chainalysis Inc., 2023.
- [3] Y. Sun et al., “GPTScan: Detecting Logic Vulnerabilities in Smart Contracts by Combining GPT with Program Analysis,” in *Proc. ICSE*, 2024.
- [4] D. David et al., “Do You Still Need a Manual Smart Contract Audit?” arXiv:2306.12338, 2023.
- [5] N. Perry et al., “Do Users Write More Insecure Code with AI Assistants?” in *Proc. CCS*, 2023.
- [6] H. Pearce et al., “Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions,” in *Proc. IEEE S&P*, 2022.
- [7] C. Tony et al., “LLMSEval: A Dataset of Natural Language Prompts for Security Evaluations,” in *Proc. MSR*, 2023.
- [8] N. Tihanyi et al., “FormAI – A Dataset of 112,000 AI-Generated C Programs with Vulnerability Classification,” arXiv:2307.02192, 2023.
- [9] J. Feist et al., “Slither: A Static Analysis Framework for Smart Contracts,” in *Proc. WETSEB*, 2019.
- [10] C. Mueller, “Mythril: Security analysis tool for EVM bytecode,” GitHub, 2018.
- [11] T. Durieux et al., “Empirical Review of Automated Analysis Tools on 47,587 Ethereum Smart Contracts,” in *Proc. ICSE*, 2020.
- [12] M. Ren et al., “Empirical Evaluation of Smart Contract Testing: What Is the Best Choice?” in *Proc. ISSTA*, 2021.
- [13] R. Schuster et al., “You Autocomplete Me: Poisoning Vulnerabilities in Neural Code Completion,” in *Proc. USENIX Security*, 2021.
- [14] D. Kang et al., “Exploiting Programmatic Behavior of LLMs: Dual-Use Through Standard Security Attacks,” in *Proc. IEEE S&P*, 2024.
- [15] G. Sandoval et al., “Lost at C: A User Study on the Security Implications of Large Language Model Code Assistants,” in *Proc. USENIX Security*, 2023.
- [16] K. Jesse et al., “Large Language Models and Simple, Stupid Bugs,” in *Proc. MSR*, 2023.
- [17] P. Tsankov et al., “Securify: Practical Security Analysis of Smart Contracts,” in *Proc. CCS*, 2018.
- [18] L. Luu et al., “Making Smart Contracts Smarter,” in *Proc. CCS*, 2016.
- [19] T. Durieux et al., “SmartBugs: A Framework to Analyze Solidity Smart Contracts,” in *Proc. ASE*, 2020.
- [20] Y. Zhang et al., “SmartLLMSentry: LLM-based Smart Contract Vulnerability Detection,” arXiv, 2024.
- [21] Z. Liu et al., “PropertyGPT: LLM-driven Formal Verification of Smart Contracts through Retrieval-Augmented Property Generation,” arXiv, 2024.
- [22] W. Chen et al., “ContractTinker: LLM-Empowered Vulnerability Repair for Real-World Smart Contracts,” arXiv, 2024.
- [23] F. Perez and I. Ribeiro, “Ignore This Title and HackAPrompt: Exposing Systemic Weaknesses of LLMs through a Global Scale Prompt Hacking Competition,” arXiv:2311.16119, 2023.
- [24] Y. Wan et al., “Poisoning Language Models During Instruction Tuning,” in *Proc. ICML*, 2023.